ACTIVIDAD 3 – SimuPi – Node-RED – MQTT en AWS

INFORME DE ACTIVIDAD

1) Se deberá entregar un documento en formato PDF, en el que se explique detalladamente los pasos que se han seguido para resolver las actividades propuestas. Dicho documento debe incluir las capturas de pantalla necesarias en las que se pueda ver el trabajo del alumno (instancias en ejecución, grupos de seguridad, SimuPi, flujos de Node-RED y dashboard). Además, se deberá entregar un **VIDEO MP4** mostrando el funcionamiento del sistema, con el dashboard interactuando con SimuPi y el flujo de Node-RED con nodos "debug" visibles.
2) Junto con el informe se entregarán el **flujo de Node-RED exportado (JSON)** y, si se desarrolló un escenario propio, el **JSON del escenario** de SimuPi.
3) Plazo de entrega: **16/10/2026 hasta las 24hs.**

EXPLICACIÓN PREVIA – EJEMPLO GUIADO

Antes de comenzar se muestra, a modo de ejemplo, un dashboard mínimo con **dos sensores** (temperatura y humedad) y **dos actuadores** (ventilador y bomba) del escenario `greenhouse`, junto con el flujo de Node-RED que lo genera. Es la base sobre la que cada grupo desarrollará su propio trabajo.



Figura 2. Dashboard de ejemplo (`http://<IP pública de B>:1880/dashboard/invernadero`).

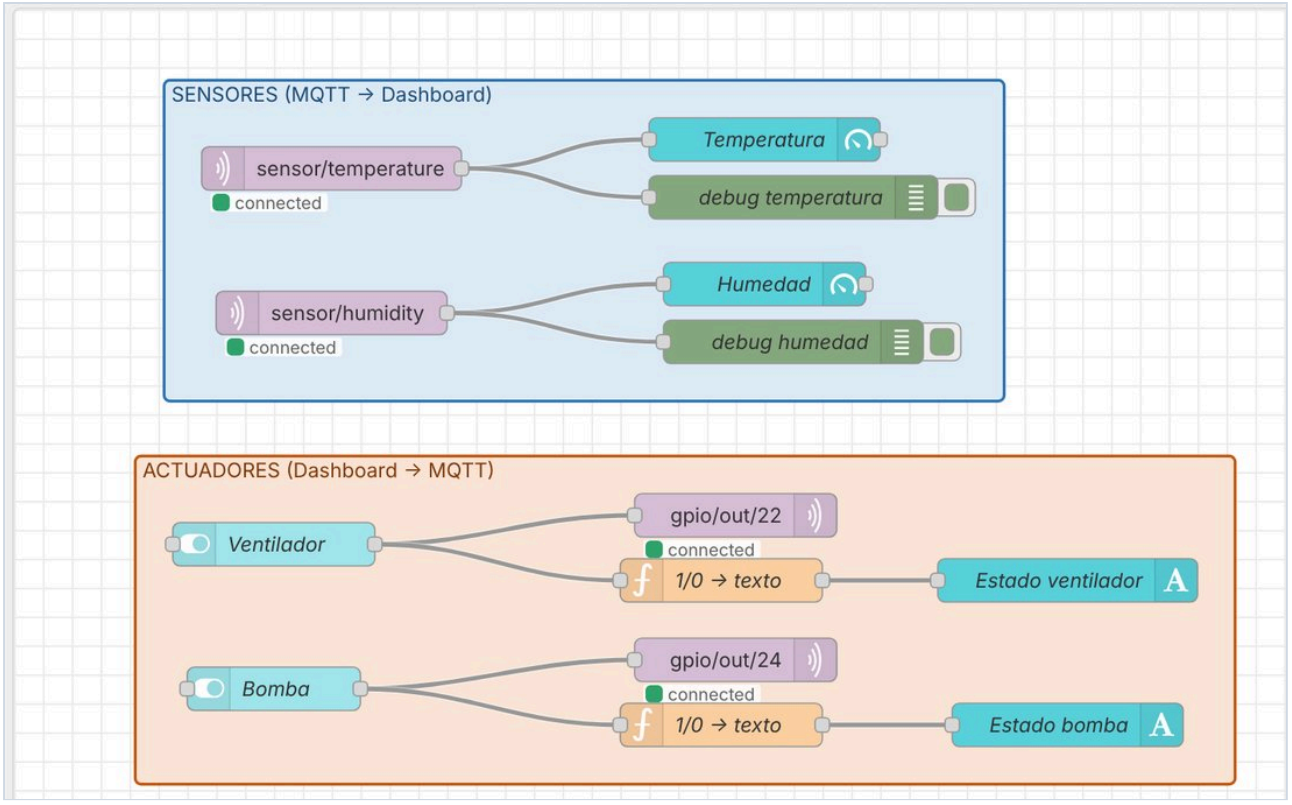
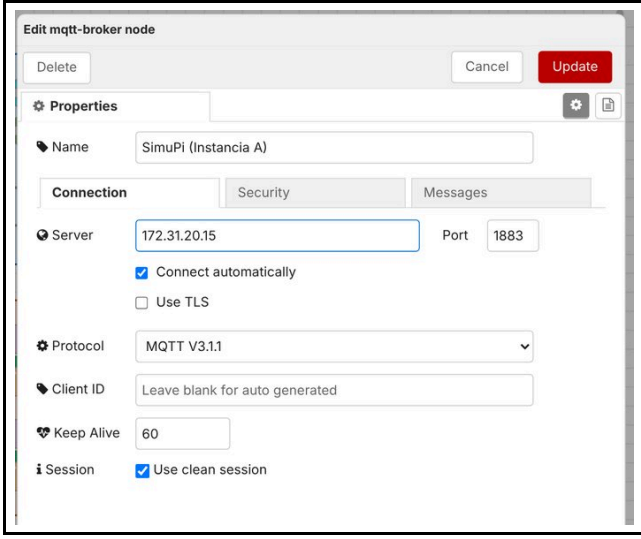
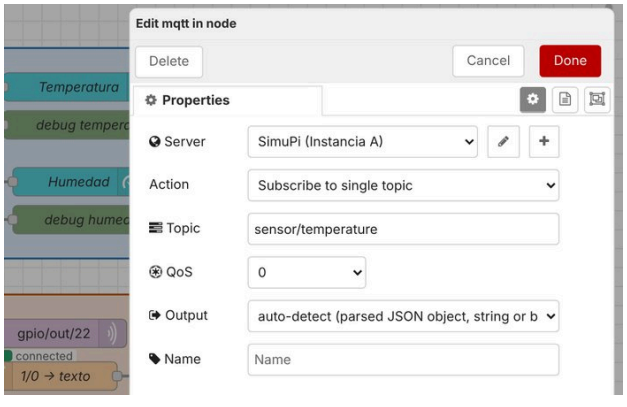


Figura 3. Flujo de Node-RED del dashboard. Arriba: sensores (MQTT → Dashboard); abajo: actuadores (Dashboard → MQTT).

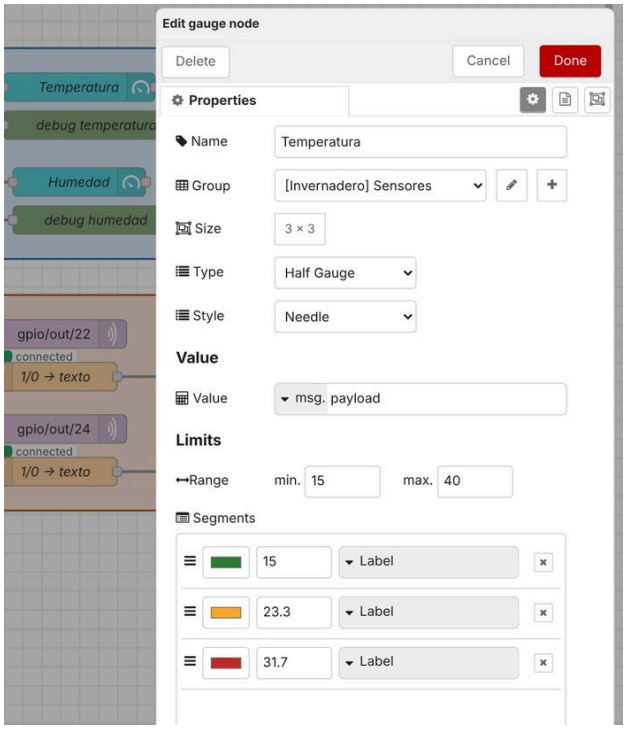
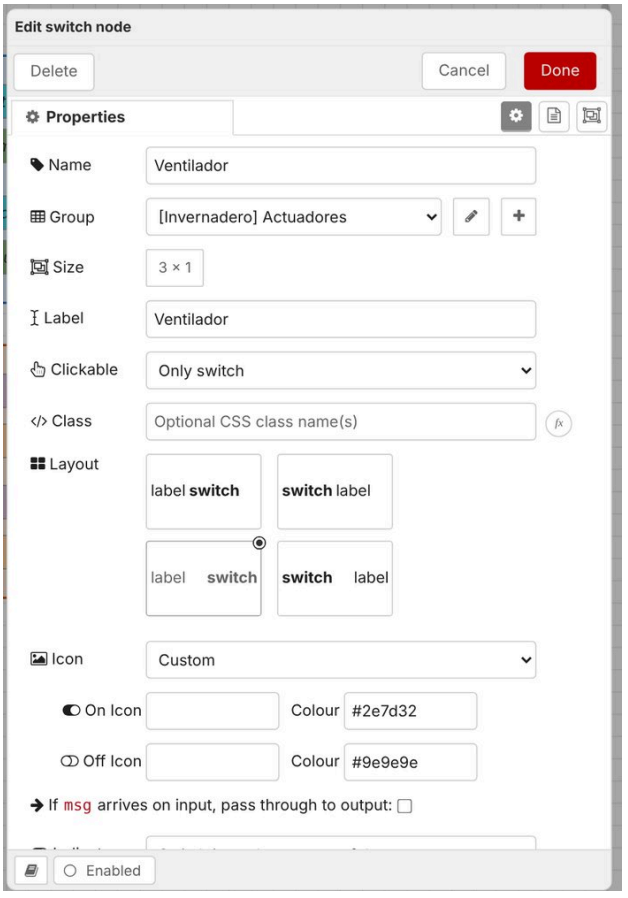
Cómo funciona el flujo

- **Sensores:** un nodo **mqtt in** por sensor se suscribe a `sensor/temperature` y `sensor/humidity`. Cada mensaje llega como número y se envía a un nodo **gauge** (indicador) y a un nodo **debug** para verificar el dato.
- **Actuadores:** cada nodo **switch** emite `"1"` (encendido) o `"0"` (apagado) hacia un nodo **mqtt out** que publica en `gpio/out/22` (ventilador) o `gpio/out/24` (bomba). SimuPi recibe el mensaje y cambia el pin.
- **Estado:** una función "1/0 → texto" convierte el comando enviado en ENCENDIDO / APAGADO y lo muestra en un nodo **text**, ya que SimuPi no publica el estado de las salidas por MQTT.

Configuración de los nodos principales

Bróker MQTT (config. compartida)	mqtt in (sensor)
	

En *Server* se coloca la **IP privada de la Instancia A** (aquí un ejemplo: `172.31.20.15`), puerto **1883**. Los nodos "mqtt in" y "mqtt out" comparten ese mismo bróker.

gauge (Temperatura)	switch (Ventilador)
	

En el **switch** se configuran *On value* = `"1"` y *Off value* = `"0"` (tipo string) y se deja el nodo conectado a un "mqtt out" con el tópico `gpio/out/22`. El **gauge** toma `msg.payload` y su rango (min/max) debe coincidir con el del sensor.

EJEMPLOS AVANZADOS DE REFERENCIA

Además del ejemplo mínimo se muestran dos dashboards más completos, que combinan visualización, control manual y **lógica de automatización**. Sirven para que cada grupo vea hasta dónde se puede llegar y qué componentes de Node-RED usar; **no son la solución de la actividad**: se espera un diseño propio, y especialmente un escenario propio en la Parte 3.

Ejemplo 2 – Invernadero automático (escenario `greenhouse`)

- **Sensores:** gauges de temperatura y humedad y un gráfico histórico con ambas series.
- **Modo MANUAL / AUTOMÁTICO:** un interruptor guarda el modo en el contexto del flujo. En manual, los switches comandan `gpio/out/22` (ventilador) y `gpio/out/24` (bomba); en automático lo hace la lógica.
- **Histéresis:** el ventilador se enciende sobre 30 °C y se apaga bajo 27 °C; la bomba se enciende bajo 40 % de humedad y se apaga sobre 55 %. Solo se publica cuando el estado cambia.
- **Alarma** de temperatura (> 35 °C) y estado de las entradas `gpio/in/17` y `gpio/in/18`.

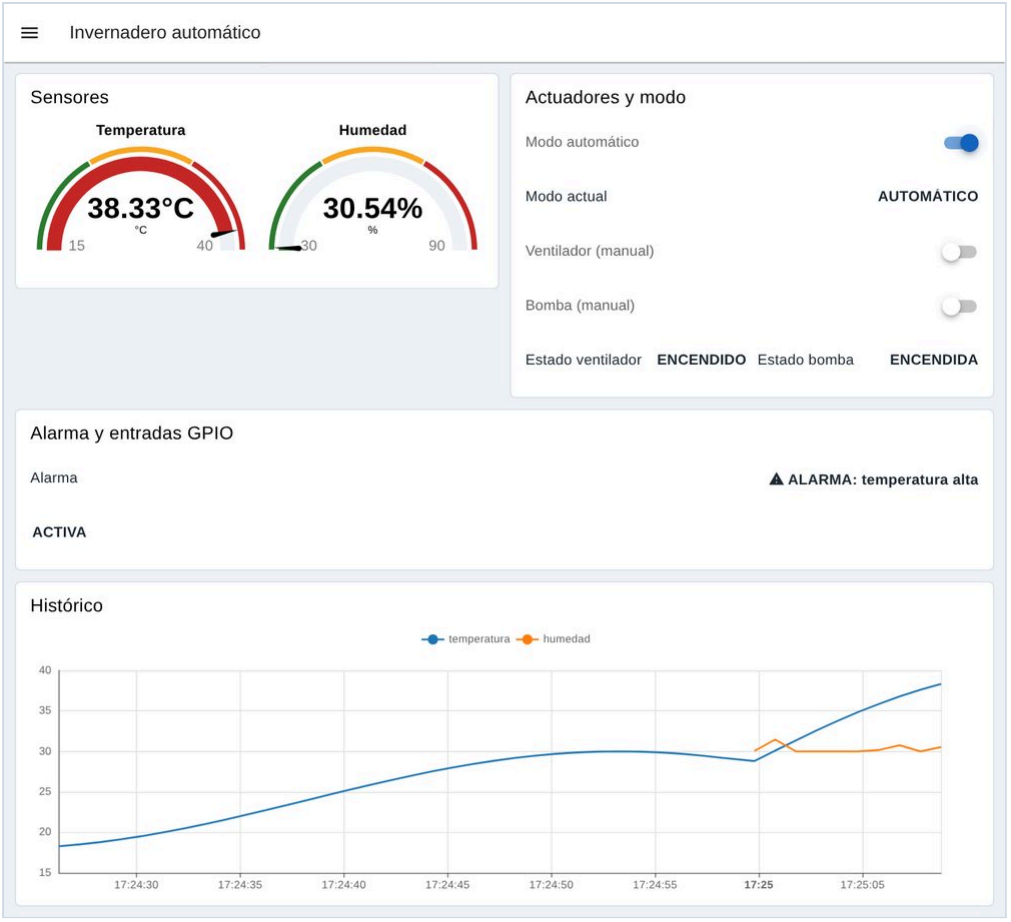


Figura 4. Dashboard “Invernadero automático” (modo automático, ventilador y bomba encendidos, alarma activa y entrada GPIO 17 accionada desde SimuPi).

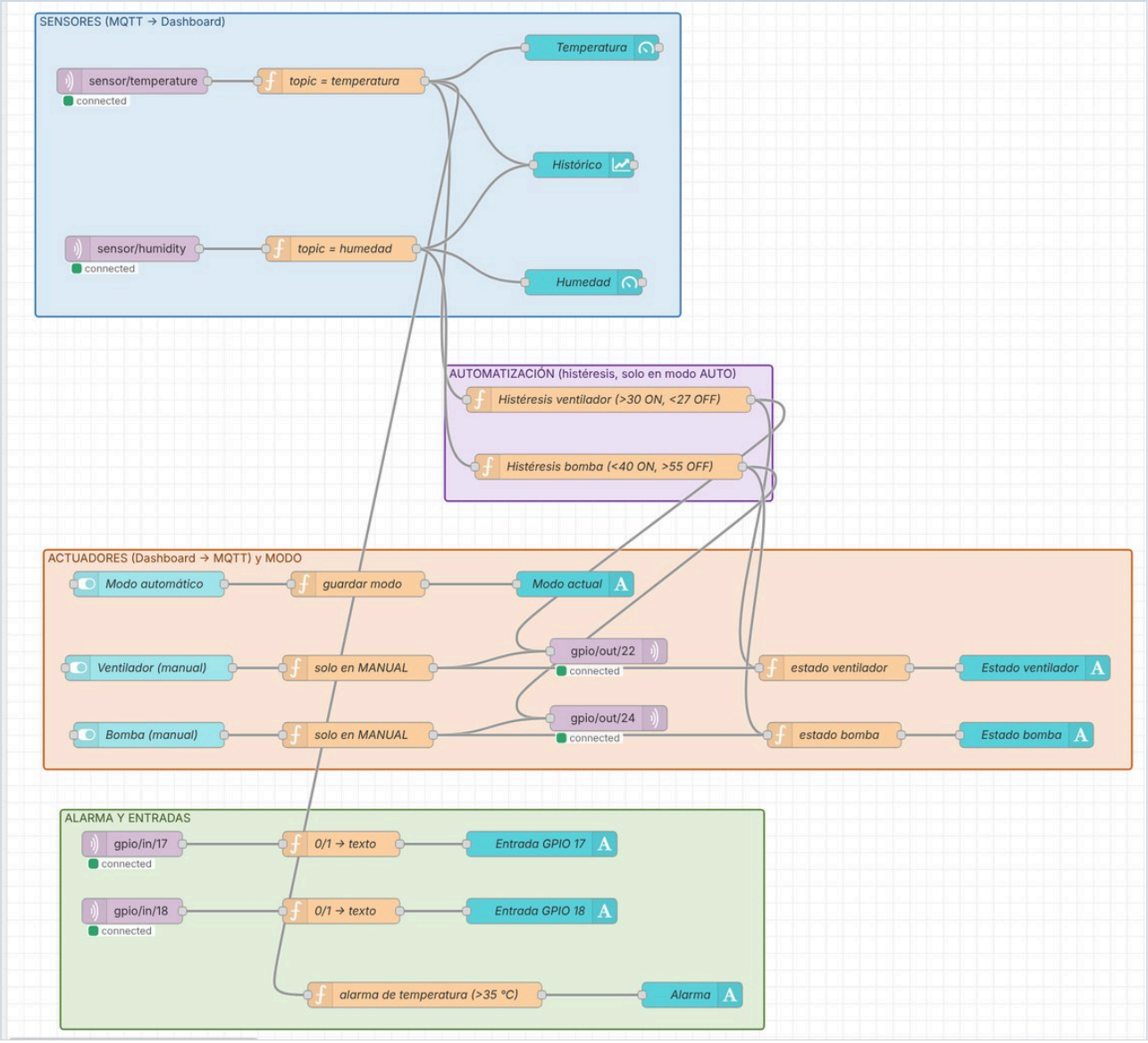


Figura 5. Flujo del ejemplo 2: sensores, automatización con histéresis, actuadores con modo manual y alarma/entradas.

Ejemplo 3 – Casa inteligente (escenario `smart_home`)

- **Sensores:** gauges de temperatura y consumo (`sensor/power`) y gráfico de temperatura.
- **Termostato:** un **slider** define la consigna (18–30 °C); si el termostato automático está activo, el calefactor (`gpio/out/24`) se enciende por debajo de la consigna −1 °C y se apaga por encima de la consigna +1 °C.
- **Alarma de consumo:** si el consumo supera 2000 W se activa `gpio/out/25` y se informa en el dashboard.
- **Luz del living** con un switch (`gpio/out/22`).

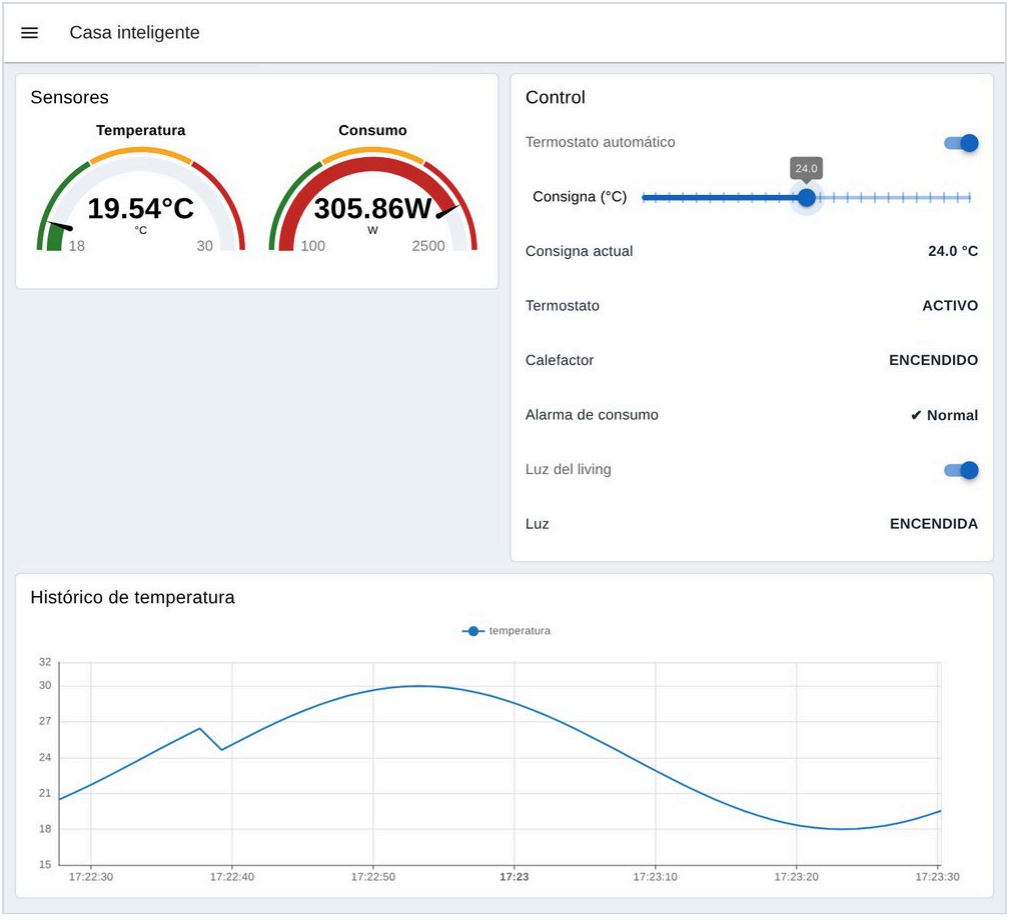


Figura 6. Dashboard “Casa inteligente”.

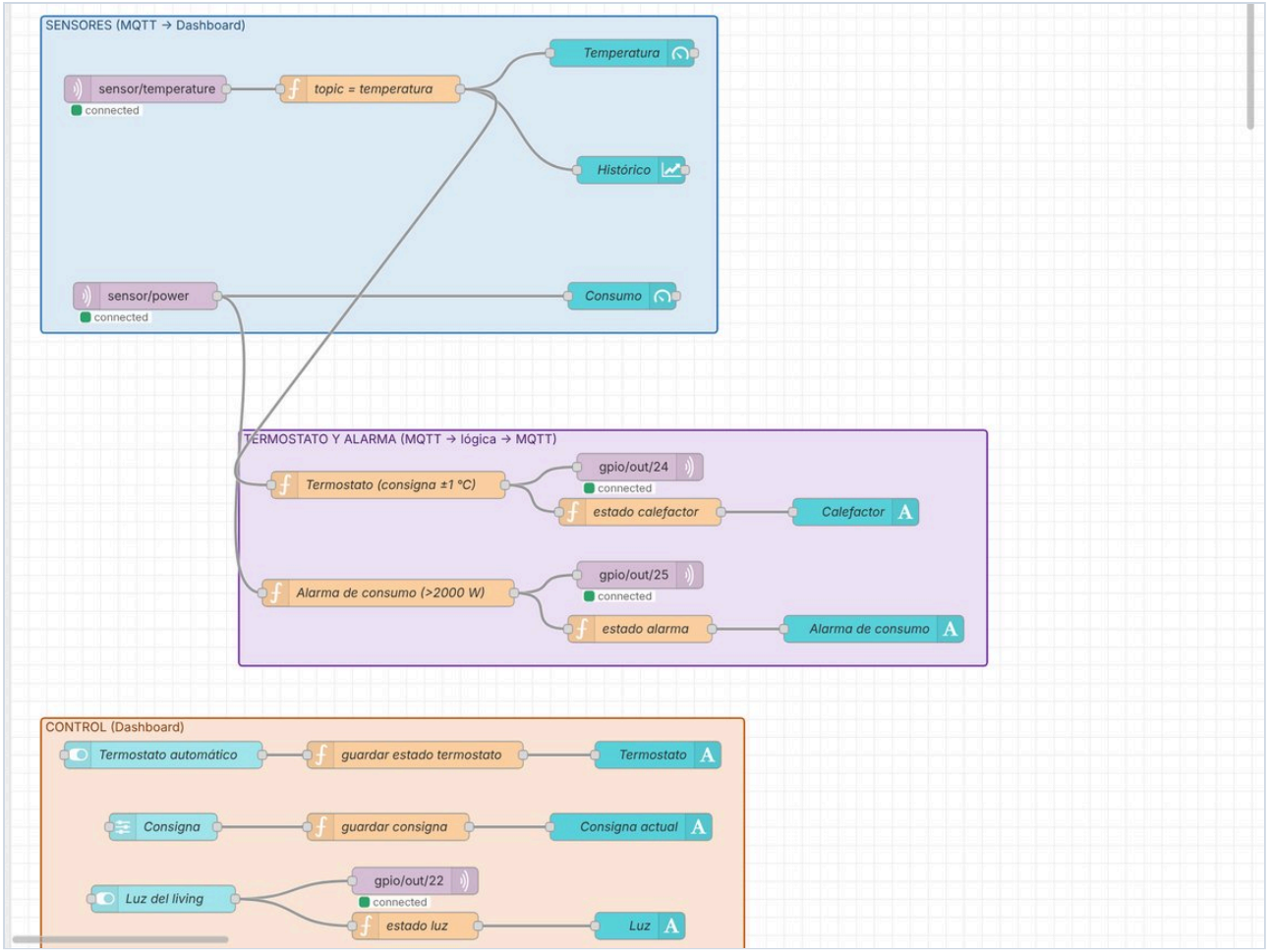


Figura 7. Flujo del ejemplo 3: sensores, termostato y alarma, y controles del dashboard.

ACTIVIDAD PROPUESTA

En esta actividad se reemplaza el bróker de AWS IoT por el **bróker MQTT embebido de SimuPi**, un simulador de Raspberry Pi presentado en clase que genera sensores, pines GPIO y mensajes MQTT. Se trabaja con dos instancias: en la **Instancia A** corre SimuPi y en la **Instancia B** corre Node-RED con un dashboard que lee y controla la simulación mediante MQTT.

> **Material de la cátedra – repositorio de SimuPi:** `https://github.com/seguren/simupi`
> En el repositorio se encuentran: el ejecutable para Linux (`bin/simupi-linux-amd64`) y para Windows, los escenarios (`scenarios/`), el *Manual de usuario* y esta actividad (`docs/`).

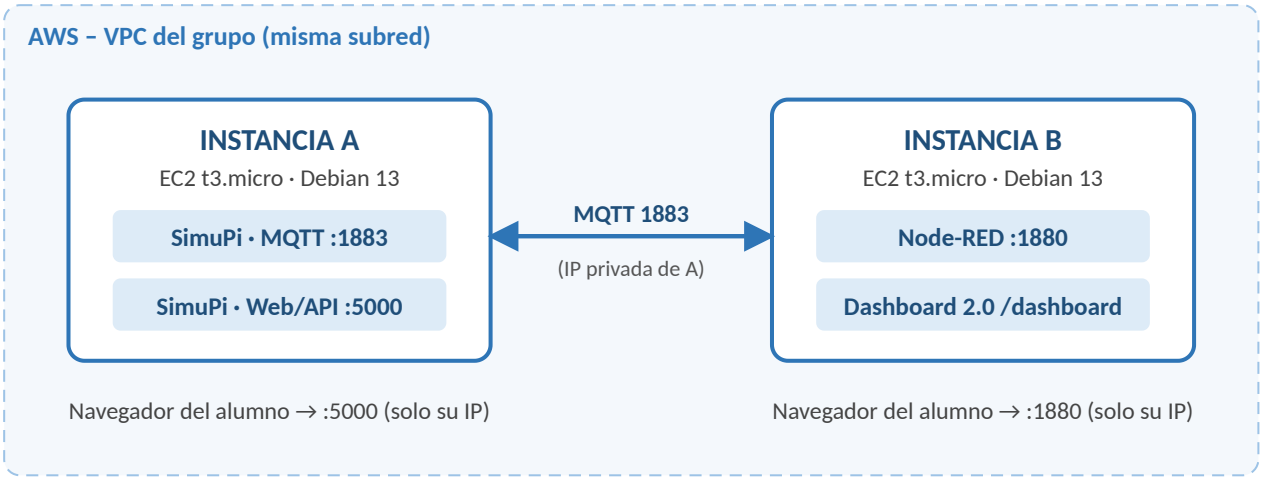


Figura 1. Arquitectura de la actividad.

Parte 1 – Infraestructura COMÚN

1) Desplegar dos instancias **t3.micro** con **Linux Debian 13** en AWS EC2: **Instancia A** (SimuPi) e **Instancia B** (Node-RED). Ambas deben estar en la misma VPC y subred.
2) Configurar los grupos de seguridad de modo que:
   - SSH (22) se permita únicamente desde la IP del alumno.
   - En A: el puerto **1883** (MQTT) se permita solo desde el grupo de seguridad de B, y el **5000** (web/API de SimuPi) solo desde la IP del alumno.
   - En B: el puerto **1880** (Node-RED y dashboard) se permita solo desde la IP del alumno.

   No se deben abrir los puertos 1883 ni 5000 a `0.0.0.0/0`: el bróker de SimuPi no tiene usuario ni contraseña.
3) En la Instancia A, descargar el ejecutable de **SimuPi para Linux (amd64)** desde el repositorio de la cátedra (`wget -O simupi https://github.com/seguren/simupi/raw/main/bin/simupi-linux-amd64 && chmod +x simupi`) y dejarlo **corriendo** aunque se cierre la sesión SSH (por ejemplo: `nohup ./simupi > simupi.log 2>&1 &`, o dentro de `tmux` / `screen`). Verificar con `curl localhost:5000/api/health` y desde la interfaz web.
4) En la Instancia B, instalar “Node-RED” y el nodo “@flowfuse/node-red-dashboard”, como se hizo en la Actividad 2.
5) Conectar B con A: configurar en Node-RED el bróker MQTT con la **IP privada de la Instancia A**, puerto 1883. Comprobar con nodos “mqtt in” y “debug” que llegan los tópicos `sensor/#` y `gpio/in/#` .

Parte 2 – Dashboard con los escenarios preconfigurados BASE

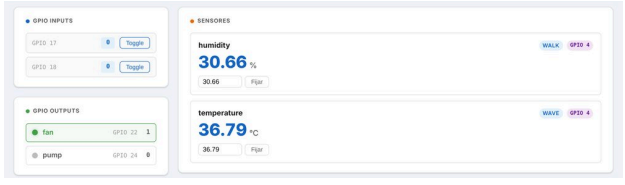
6) Desarrollar, en la Instancia B, un dashboard que funcione con al menos uno de los escenarios que trae SimuPi (`garage` , `greenhouse` o `smart_home`) y que permita:
   - Visualizar el valor actual de cada sensor (indicador o gauge) y su evolución en el tiempo (gráfico).
   - Controlar los **GPIO outputs** con interruptores que publiquen en `gpio/out/<pin>` (por ejemplo, ventilador y bomba en `greenhouse`).
   - Mostrar el **estado de los GPIO inputs** (`gpio/in/<pin>`) y de los **GPIO outputs**.
7) Cambiar de escenario desde la interfaz web de SimuPi (puerto 5000) y adaptar el dashboard para que funcione con **al menos dos escenarios** distintos, sin indicadores con datos viejos.

Parte 3 – Escenario propio MAYOR PONDERACIÓN

8) Diseñar un **escenario original** para SimuPi sobre un caso de uso elegido por el grupo (ver ideas más abajo) que tenga como mínimo: **3 sensores** con al menos **2 modos de simulación distintos** y parámetros realistas (rango, unidad, período), **2 GPIO outputs** y **1 GPIO input**, todos con alias significativos. Crearlo desde la ventana *Configuración → Guardar como escenario* y verificar que aparece en el selector.
9) Desarrollar un **dashboard específico** para ese escenario, con diseño y textos acordes al caso de uso, y una **lógica de automatización** en Node-RED que incluya al menos:
   - Una regla con **umbral e histéresis** que accione un GPIO output (por ejemplo, encender un ventilador sobre 30 °C y apagarlo bajo 27 °C).
   - Una **alarma** visible en el dashboard cuando un valor sale de su rango seguro.
   - Un **modo MANUAL / AUTOMÁTICO** seleccionable desde el dashboard.
10) Documentar en el informe: descripción del caso de uso, tabla de sensores (modo, rango, unidad), pines y tópicos MQTT, y las reglas de automatización implementadas.

Ideas de escenarios: cámara frigorífica, riego automático, tanque de agua con bomba, estacionamiento con barrera, invernadero con ventilación y alarma, galpón avícola, sala de servidores con refrigeración.

Ejemplo (Parte 2, escenario `greenhouse` , ventilador encendido desde el dashboard):

INSTANCIA A – SimuPi – GRUPO XX - 2026	INSTANCIA B – Node-RED – GRUPO XX - 2026
	

El ejemplo es solo orientativo: cada grupo debe diseñar su propio dashboard y, para la Parte 3, su propio escenario.

IMPORTANTE: Una vez resuelta la actividad, se deberán **terminar todas las instancias utilizadas** (Instancia A e Instancia B) para evitar costos innecesarios.

GUÍA RÁPIDA

Tópicos MQTT de SimuPi (bróker en la Instancia A, puerto 1883, sin usuario ni contraseña)

Sentido	Tópico	Contenido
SimuPi → Node-RED	`sensor/<nombre>`	Valor numérico con 2 decimales, por ejemplo `24.53` (una lectura por intervalo).
SimuPi → Node-RED	`gpio/in/<pin>`	`0` o `1`.
Node-RED → SimuPi	`gpio/out/<pin>`	`0` o `1` para apagar o encender una salida.

> **Atención:** SimuPi **no publica por MQTT el estado de los GPIO outputs**. Para mostrarlo en el dashboard se refleja el **último comando enviado**, como en el ejemplo de la explicación previa. El estado real se puede comprobar en la interfaz web de SimuPi.

API REST de SimuPi (referencia)

SimuPi expone una API REST en el puerto **5000** (`http://<IP de A>:5000`), la misma que usa su interfaz web. La actividad se resuelve solo con MQTT, pero la API es útil para probar, automatizar y depurar. Las respuestas son JSON; los errores devuelven `{"error": "mensaje"}` con código `400` (datos inválidos) o `404` (no existe). Los cuerpos se envían con `Content-Type: application/json`. Los nombres de pines y sensores son los del escenario activo.

Método	Ruta	Descripción, parámetros y respuesta
Sistema y estado		
GET	`/api/health`	Comprueba que el servidor responde. Devuelve `{"status":"ok"}`.
GET	`/api/state`	Estado completo: `scenario`, `sensors` (valor actual), `sensor_units`, `sensor_modes`, `sensor_limits` (min/max), `sensor_pins`, `gpio_inputs`, `gpio_outputs`, `aliases`, `failures` y `overrides` (valores fijados).
Escenarios		
GET	`/api/scenarios`	Lista los escenarios disponibles (incluidos y guardados por el usuario).
GET	`/api/scenarios/<nombre>`	Devuelve la definición JSON de un escenario (sensores, pines, alias). `404` si no existe.
POST	`/api/scenarios/load/<nombre>`	Carga el escenario en caliente: reinicia sensores y GPIO y borra historial, valores fijados y fallas. Respuesta: `{"success":true,"scenario":"..."}`
Configuración		
GET	`/api/config`	Configuración del escenario activo: `inputs`, `outputs`, `sensors` y `aliases`.
POST	`/api/config/apply`	Aplica una configuración (mismo formato que `/api/config`) al sistema en ejecución, sin guardarla.

POST	`/api/config/save/<nombre>`	Aplica y **guarda** la configuración como escenario propio. El nombre admite letras, números, `-` y `_`.
GPIO inputs		
POST	`/api/gpio/<pin>`	Fija una entrada. Cuerpo: `{"value": 1}`. Publica en `gpio/in/<pin>`. Error si el pin no es una entrada.
POST	`/api/gpio/<pin>/toggle`	Invierte el valor de la entrada (0 ↔ 1) y lo publica por MQTT.
POST	`/api/gpio/<pin>/pulse`	Pone la entrada en 1 y vuelve a 0 luego de unos segundos. Cuerpo opcional: `{"duration": 0.5}` (por defecto 0,5 s).
Sensores		
POST	`/api/sensor/<nombre>`	Fija el valor del sensor, ignorando su modo de simulación (en modo `slider` solo actualiza el valor). Cuerpo: `{"value": 25.0}`.
DELETE	`/api/sensor/<nombre>`	Libera el valor fijado y el sensor retoma su simulación.
POST	`/api/failure/<sensor>`	Simula una falla. Cuerpo: `{"mode": "stuck"}` (valor congelado), `"offline"` (deja de publicar) o `"clear"` (quita la falla).
Historial		
GET	`/api/history`	Historial de todos los sensores: objeto con una lista de lecturas por sensor.
GET	`/api/history/<sensor>`	Últimas lecturas del sensor (hasta 500): lista de `{"timestamp", "value"}`.
GET	`/api/export/<sensor>`	Descarga el historial del sensor en CSV (`timestamp,value`).

Las salidas (`gpio/out/<pin>`) **no** tienen endpoint: se comandan únicamente por MQTT, y su estado se consulta en el campo `gpio_outputs` de `/api/state`.

Ejemplos con `curl`:

```
curl http://<IP de A>:5000/api/state
curl -X POST http://<IP de A>:5000/api/scenarios/load/smart_home
curl -X POST http://<IP de A>:5000/api/sensor/temperature \
     -H "Content-Type: application/json" -d '{"value": 31.5}'
curl -X POST http://<IP de A>:5000/api/gpio/17/toggle
```

Plantilla de escenario propio

```
{
  "inputs":  [17],
  "outputs": [22, 24],
  "aliases": { "17": "puerta", "22": "ventilador", "24": "alarma" },
  "sensors": {
    "temperatura": {
      "mode": "wave", "min": 2, "max": 12,
      "interval": 1, "period": 90, "unit": "°C"
    },
    "humedad": {
      "mode": "walk", "min": 40, "max": 90,
      "interval": 1, "step": 1.5, "unit": "%"
    },
    "nivel": {
      "mode": "sawtooth", "min": 0, "max": 100,
      "interval": 1, "period": 120, "unit": "%"
    }
  }
}
```

Modos disponibles: `wave`, `triangle`, `sawtooth`, `square`, `walk`, `ramp`, `random` y `slider`. En todo sensor, `min` debe ser menor que `max`. El nombre del escenario solo admite letras, números, guiones y guiones bajos. Los detalles de la interfaz están en el *Manual de usuario de SimuPi*.